

Java Collections

The Java Collections Framework is Java's standard library for groups of objects: the interfaces (List, Set, Queue, Map), their implementations (ArrayList, HashSet, HashMap, and the rest), and reusable algorithms over them. It lives in `java.util`.

1. What is the Java Collections Framework?

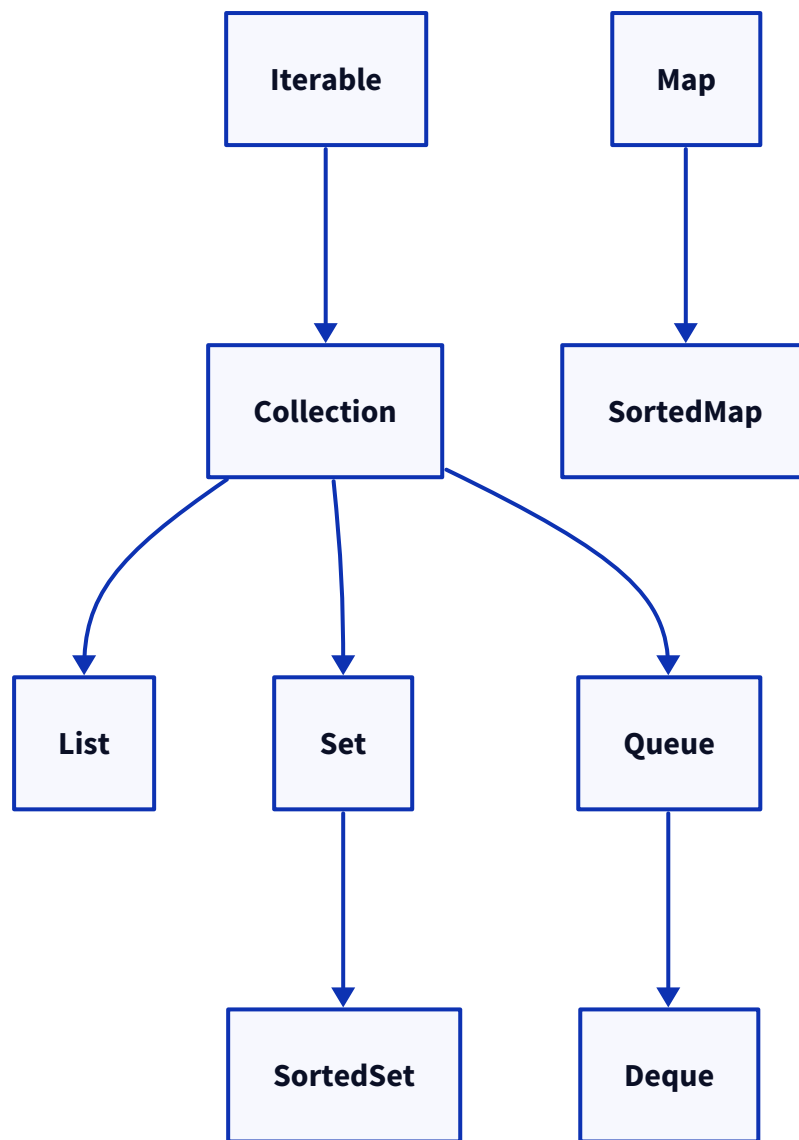
It is Java's standard set of types for **groups of objects**: a family of interfaces (List, Set, Queue, Map), concrete implementations (ArrayList, HashSet, HashMap, and so on), and algorithms (sort, search, shuffle) in the `Collections` utility class. Together they give one consistent way to store, retrieve, and manipulate data.

2. Why was the Collections Framework introduced?

Before it, Java had `Vector`, `Hashtable`, and plain arrays, each with its own unrelated API, so code was not interoperable and you rewrote the same logic everywhere. The framework gave **one consistent set of interfaces**, algorithms that work across any implementation, and the freedom to swap one implementation for another without touching the rest of the code.

3. Collections Framework architecture and hierarchy.

There are **two roots**. `Iterable` sits at the top, `Collection` extends it, and `List`, `Set`, and `Queue` extend `Collection`. **Map is separate**, it is not a `Collection`, because it holds key-value pairs rather than single elements. Sorted variants (`SortedSet`, `NavigableSet`, `SortedMap`, `NavigableMap`) and `Deque` (which extends `Queue`) fill out the tree.



The main interfaces:

- **List**: ordered, indexed, allows duplicates.
- **Set**: no duplicates.
- **Queue / Deque**: ordered for processing (FIFO, LIFO, or priority).
- **Map**: unique keys to values.

4. Collection v/s Collections.

`Collection` (singular) is the root **interface** for `List`, `Set`, and `Queue`. `Collections` (plural) is a **utility class** of static helpers like `sort`, `reverse`, `unmodifiableList`, and `synchronizedMap`. One is a type; the other is a toolbox of methods that operate on that type.

5. Collection v/s Map.

A `Collection` holds **single elements**; a `Map` holds **key-value pairs** with unique keys. `Map` is deliberately not part of the `Collection` interface, which is why you cannot iterate a `Map` directly, you iterate its `keySet()`, `values()`, or `entrySet()`.

6. How do you choose the right collection for a problem?

Match the collection to what the problem actually needs:

Need	Use
Indexed access, mostly reads	<code>ArrayList</code>
Frequent add/remove at the ends	<code>ArrayDeque</code> (or <code>LinkedList</code>)
No duplicates, fast lookup	<code>HashSet</code>
No duplicates, sorted	<code>TreeSet</code>
Key to value, fast lookup	<code>HashMap</code>
Key to value, sorted by key	<code>TreeMap</code>
Preserve insertion order	<code>LinkedHashMap</code> / <code>LinkedHashSet</code>
FIFO processing	<code>ArrayDeque</code> as a queue
Priority processing	<code>PriorityQueue</code>
Thread-safe map	<code>ConcurrentHashMap</code>

7. List: hierarchy and implementations, with example.

`List` extends `Collection` and is an **ordered, indexed** collection that allows duplicates. The implementations:

- **ArrayList**: resizable array. $O(1)$ indexed get, $O(1)$ amortized append, $O(n)$ insert or remove in the middle. The default choice.
- **LinkedList**: doubly-linked list. $O(1)$ insert and remove at the ends, but $O(n)$ indexed access. Also implements `Deque`.
- **Vector / Stack**: legacy, synchronized on every method; avoid them. Use `ArrayList`, or `ArrayDeque` for a stack.

```
List<String> names = new ArrayList<>();
names.add("Priya");
names.add("Rohit");
names.get(0);      // "Priya"
```

8. Set: hierarchy and implementations, with example.

`Set` extends `Collection` and holds **no duplicates**. The implementations:

- **HashSet**: hash table, $O(1)$ add and contains, no order.
- **LinkedHashSet**: hash table plus a linked list, preserves insertion order.
- **TreeSet**: red-black tree, keeps elements sorted, $O(\log n)$, implements `NavigableSet`.

```
Set<Integer> seen = new HashSet<>();
seen.add(1);
seen.add(1);           // stored once
seen.contains(1);      // true
```

9. Queue and Deque: hierarchy and implementations, with example.

`Queue` extends `Collection` and orders elements for processing; `Deque` extends `Queue` and is double-ended. The implementations:

- **ArrayDeque**: resizable circular array, the default queue and stack, $O(1)$ at both ends, no nulls.
- **PriorityQueue**: binary min-heap, polls the smallest by natural order or a `Comparator`, $O(\log n)$.
- **LinkedList**: also serves as a `Queue` and `Deque`.
- **BlockingQueue family** (`ArrayBlockingQueue`, `LinkedBlockingQueue`): concurrent, block when full or empty, for producer-consumer.

Use a `Deque` for a stack (`push` / `pop`) rather than the legacy `Stack`.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
stack.pop();           // 2 (LIFO)

Queue<Integer> q = new ArrayDeque<>();
q.offer(1);
q.offer(2);
q.poll();              // 1 (FIFO)
```

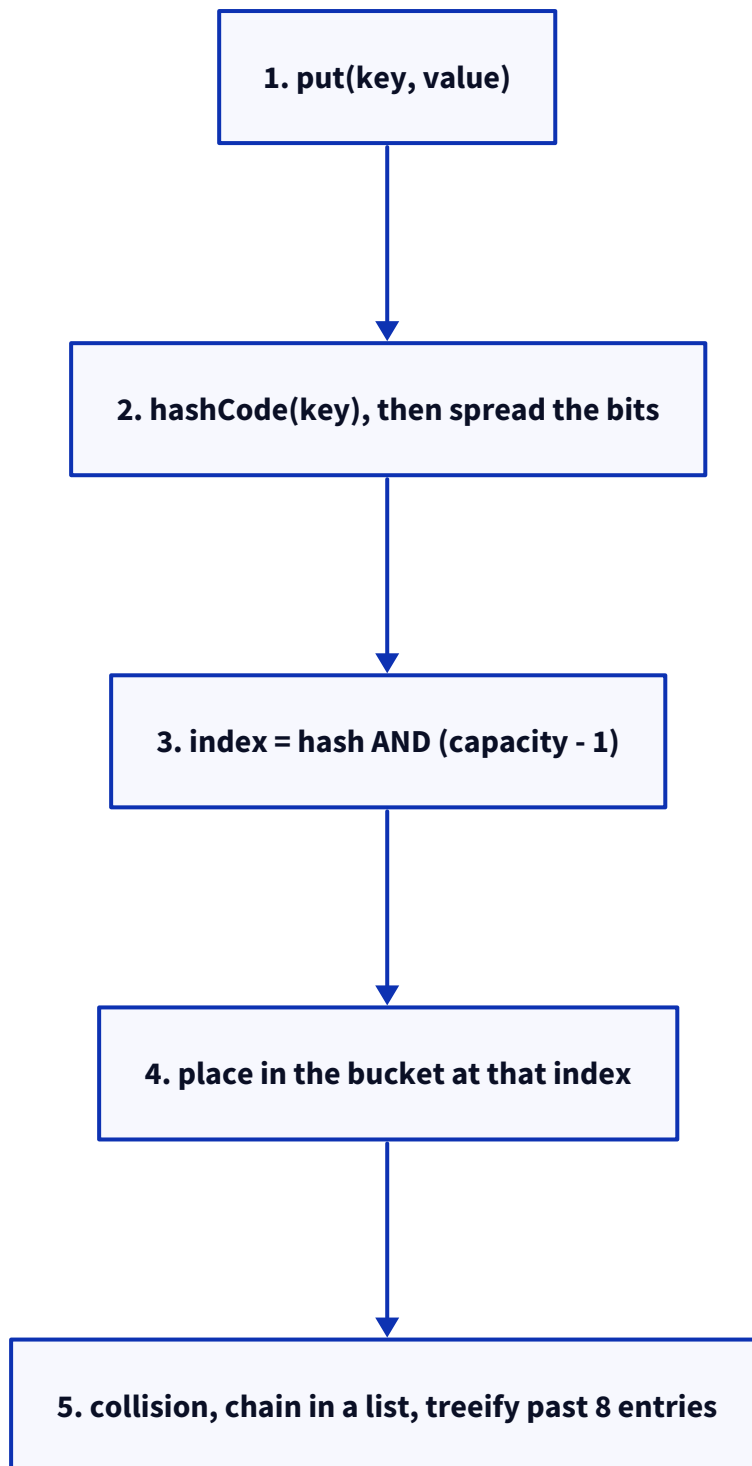
10. Map: hierarchy, implementations, and how HashMap works, with example.

`Map` (separate from `Collection`) maps unique keys to values. The implementations:

- **HashMap**: hash table, $O(1)$ average get and put, no order, allows one null key.
- **LinkedHashMap**: preserves insertion (or access) order.
- **TreeMap**: red-black tree, sorted by key, $O(\log n)$, implements `NavigableMap`.
- **Hashtable**: legacy and synchronized; use `ConcurrentHashMap` when you need thread safety.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Priya", 30);
ages.getOrDefault("Rohit", 0); // 0
```

How HashMap works. It holds an **array of buckets**. On `put`, it takes the key's `hashCode()`, spreads the bits, and maps it to a bucket index. Two different keys landing in the same bucket (a collision) chain in a **linked list**, which **converts to a balanced tree once a bucket passes 8 entries** (and the table is large enough), keeping the worst case at $O(\log n)$ instead of $O(n)$. The table **resizes (doubles)** when it passes the load factor, 0.75 by default.



11. Common mistakes developers make with Java Collections.

- **Modifying a collection while iterating it** (without the iterator's own `remove()`), which throws `ConcurrentModificationException`.
- **Reaching for `Vector`, `Stack`, or `Hashtable`** out of habit; they synchronize every call. Use `ArrayList`, `ArrayDeque`, and `HashMap`.
- **Assuming `HashMap` keeps order or is thread-safe.** It is neither, use `LinkedHashMap` for order and `ConcurrentHashMap` across threads.
- **Broken `hashCode()` / `equals()`** (or overriding one but not the other), which quietly breaks `HashMap` and `HashSet` lookups.

- **A compound op on a `synchronizedMap`** ("if absent, then put") without extra locking is still a race; use `putIfAbsent` or `compute`.
- **Not sizing a `HashMap`** when the count is known, and boxing primitives (`Integer` keys) in hot loops.

12. Common collection comparisons.

Comparison	The difference
Array v/s ArrayList	Fixed size, holds primitives, no methods; ArrayList is resizable, objects only, rich API
ArrayList v/s LinkedList	Array-backed, O(1) index, cache-friendly; LinkedList is node-backed, O(1) at ends but O(n) index. ArrayList wins in practice
Stack v/s ArrayDeque	Stack is legacy and synchronized (extends Vector); ArrayDeque is the modern, faster stack
Stack v/s Queue	LIFO v/s FIFO
Queue v/s Deque	Queue adds and removes at opposite ends (FIFO); Deque works at both ends (FIFO or LIFO)
HashSet v/s LinkedHashSet	No order v/s insertion order
HashSet v/s TreeSet	O(1) unordered v/s O(log n) sorted
LinkedHashSet v/s TreeSet	Insertion order, O(1); v/s sorted order, O(log n)
List v/s Set	Ordered, indexed, allows duplicates; v/s no duplicates, no index
Set v/s Map	Single elements v/s key-value pairs
Queue v/s List	Access at the ends for processing; v/s random indexed access

13. Concurrency basics: process, thread, concurrency, parallelism, Runnable, Callable.

Comparison	The difference
Process v/s Thread	A process has its own memory and is isolated and heavier; a thread shares the process's memory and is lighter and faster to switch
Concurrency v/s Parallelism	Concurrency is tasks making progress by interleaving (even on one core); parallelism is tasks running at the same instant on multiple cores
Runnable v/s Callable	<code>run()</code> returns nothing and cannot throw a checked exception; <code>call()</code> returns a value and can throw, and pairs with a <code>Future</code>
Thread v/s Runnable	A <code>Thread</code> is the thread of execution; a <code>Runnable</code> is the task it runs. Prefer implementing <code>Runnable</code> and handing it to a pool over extending <code>Thread</code>

14. Which Java collections do you use most frequently in production, and why?

Honestly it is **ArrayList** and **HashMap** for the vast majority of cases: a list I iterate and index, and a key-to-value lookup, both $O(1)$ for the common operations and cheap on memory. After that, **HashSet** for dedup and membership tests, **ArrayDeque** when I need a stack or a queue, and **ConcurrentHashMap** when a map is shared across threads. The rarer ones like **TreeMap** or **PriorityQueue** come out only when I specifically need sorted order or priority. *(Adapt to your own stack.)*

15. Describe a concurrency issue you encountered in production. How did you debug and resolve it?

So the one I would tell is a **lost update on a shared map**. Picture two threads doing a read-modify-write on the same key, say bumping a counter, on a plain **HashMap** or even a **synchronizedMap**: updates just quietly get lost, and worse, a plain **HashMap** can corrupt under concurrent writes and leave a reader spinning forever. The way it shows up is **counts that do not add up**, or an intermittent hang, the classic heisenbug that vanishes the moment you attach a debugger. What I would do is pull a **thread dump** to catch the spin or a deadlock, add some logging around the shared state, and then fix it by moving to a **ConcurrentHashMap with an atomic compute or merge**, or an **AtomicLong** or **LongAdder** for a counter, so the read-modify-write is atomic and the race is just gone. *(Adapt to your own example.)*

16. Have you replaced synchronized code with concurrent utilities? What improvements did you observe?

Yes. The common swap is replacing a **synchronized block around a HashMap** (or a `Collections.synchronizedMap`) with a **ConcurrentHashMap**, and a **synchronized counter** with an **AtomicLong** or **LongAdder**. The **synchronized** version funnels every access through one lock, so under contention the threads just queue up behind it. The concurrent versions use fine-grained locking or CAS, so reads do not block and writes only contend per bucket. What I saw was **higher throughput and lower tail latency under load**, and simpler code, because the manual locking goes away.

17. How would you process millions of concurrent requests while minimising contention and maximising CPU utilisation?

The way I would approach it is to keep every CPU busy with real work and off locks. A few things I would lean on:

- **Size the thread pool to the work**: around the number of cores for CPU-bound work, higher for IO-bound; never a thread per request.
- **Prefer non-blocking or async IO** so threads are not parked waiting on a socket or a database.
- **Minimise shared mutable state**, and where you must share, use **lock-free or striped structures** (`ConcurrentHashMap`, `LongAdder`) instead of one global lock.
- **Partition the work by key** so each shard is handled independently, which removes cross-request contention.
- **Batch and apply backpressure** with a bounded queue, so a surge does not exhaust memory or thrash the CPU.

The theme is to keep the CPUs busy with real work, not blocked on locks or waiting on IO.